Introduction to Computer Programming

Prof. Alessio Martino

amartino@luiss.it

Department of Business and Management
Viale Romania 32, 00197 Rome

A.Y. 2024-2025

Outline

1. Reusing Code
2. Functions
   - `print` vs. `return`
   - One return, multiple values
3. Variable Scope
   - Running Example
   - Important Caveats on Variable Scopes
   - The `global` Keyword
   - Nested Functions
4. Positional vs. Keyword Arguments
   - Optional and Default Parameters
5. Built-in Functions
6. Modules
   - Using Modules in Python
   - Common Modules in Python
   - Interactive Documentation
7. References
8. Acknowledgments

Reusing Code

So far ...

- we have covered language mechanisms
- we know how to write different `.py` files for different computations
- ... where each file contains some piece of code
- ... and each code is a sequence of operations

Problem

This approach is:

- easy for small-scale problems
- messy for large-scale problems
- hard to keep track of details and maintain
- hard to reuse inside more complex programs

Functions

More code does not necessarily mean good practice → use functions.

Why?

- functions are reusable chunks of code
- functions run if and only if are "called" or "invoked"

Characteristics of a function:

- functions have a name
- functions have (0 or more) parameters
- functions have a docstring[1]
- functions have a body
- functions return something.

[1] Optional, but recommended

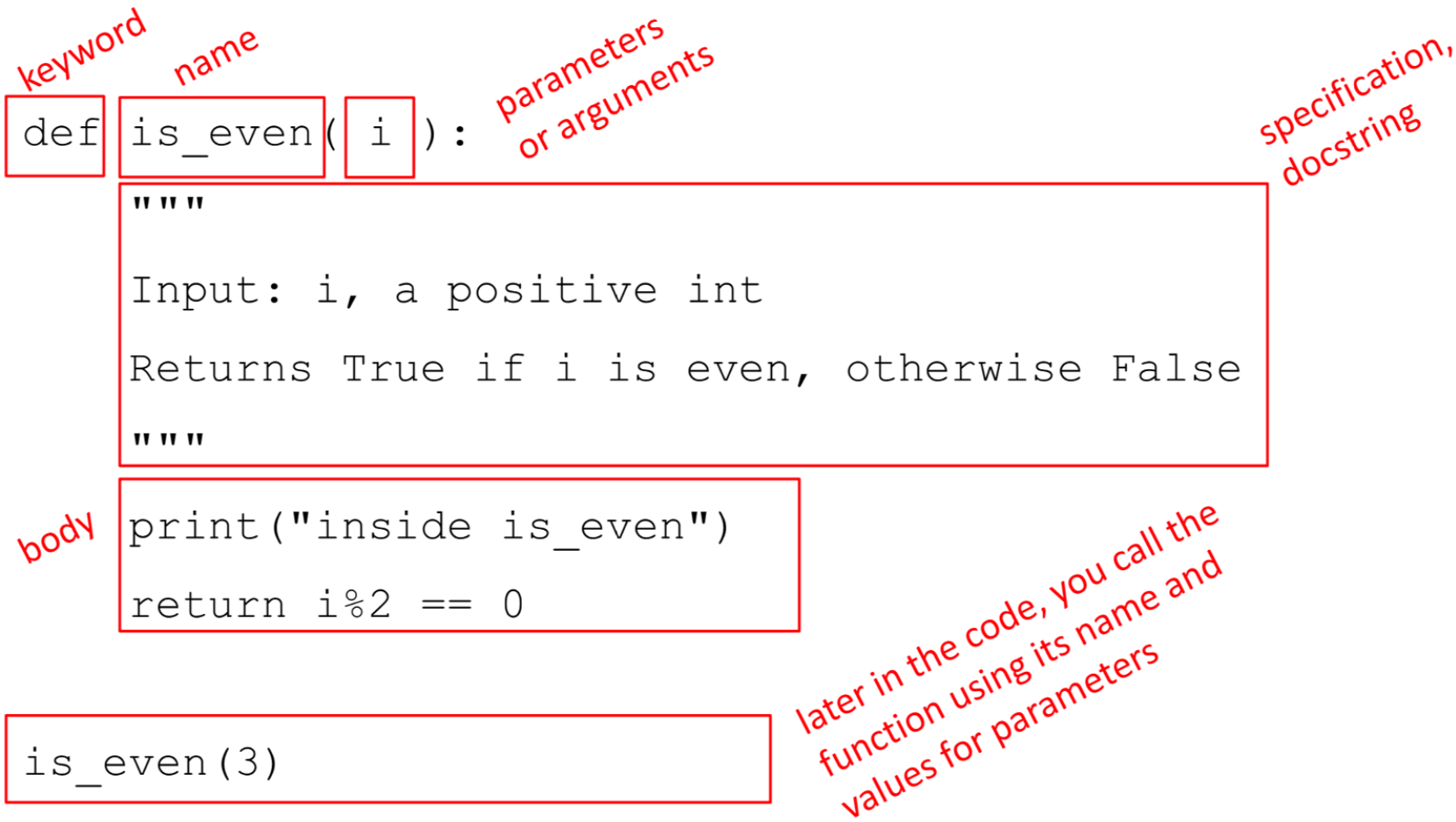

Figure 1: Anatomy of a function.

```
def is_even( i ):
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    print("inside is_even")
    return i%2 == 0
```

run some commands

keyword

expression to evaluate and return

Figure 2: Function body.

The importance of **return**:

```
def is_even( i ):
    """
    Input: i, a positive int
    Does not return anything
    """
    i%2 == 0
```

without a return statement

What happens when running e.g.

```
n=is_even(2)
```

- Python returns `None` if no **return** is given
- `None` represents the absence of a value

`print`

- can be placed anywhere (inside and/or outside functions)
- as many `print` statements as you want
- code after `print` statements is executed inside a function
- has a value associated to it, outputted to the console

`return`

- has a meaning only inside a function
- only one `return` statement inside a function
- code after `return` statement is not executed
- has a value associated to it, returned to the function caller

The **return** statement can return multiple values (if needed):

- list of values separated by commas
- returns a tuple[2]

Example:

```
def hms(nsec):
    """Takes nsec (n. of seconds) and splits in hours, minutes, seconds"""
    hh = nsec // 3600
    nsec = nsec % 3600
    mm = nsec // 60
    ss = nsec % 60
    return hh, mm, ss

print(hms(4000))
# Out: (1, 6, 40)
print(hms(100000))
# Out: (27, 46, 40)
```

[2] We will see tuples later in the course, for now we just print multiple outputs.

Variable Scope

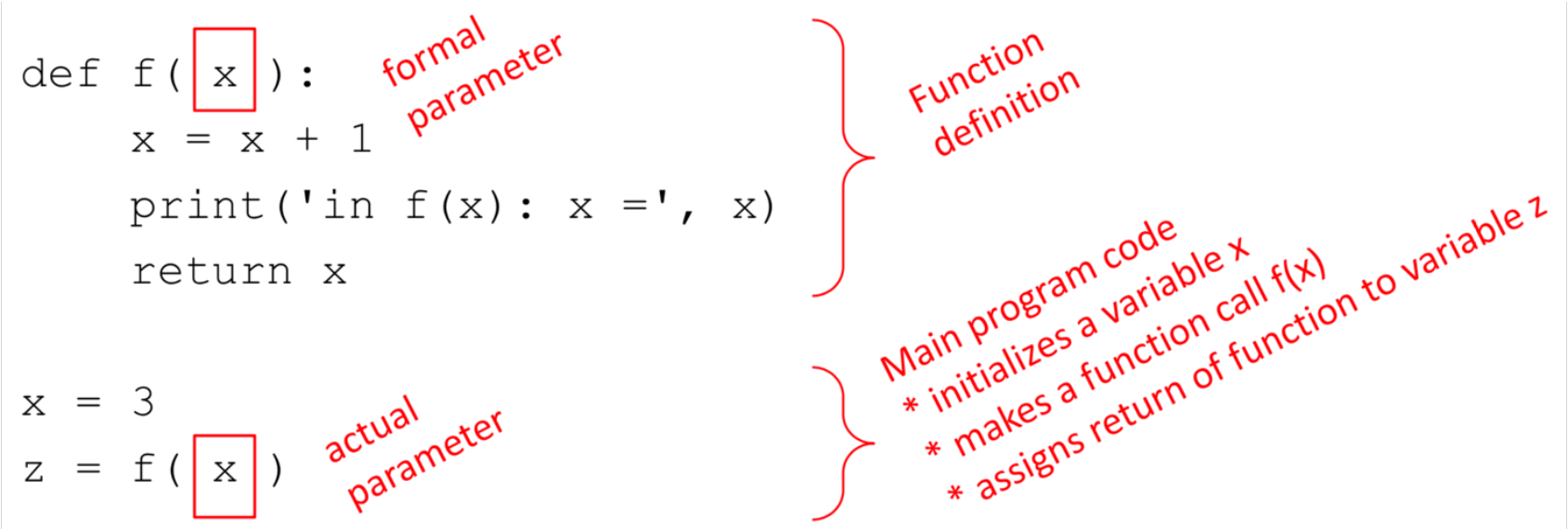

Figure 3: Variable scope (inside/outside function).

- the formal parameter gets bound to the actual parameter when the function is called
- when entering a function, a new scope (or frame, or environment) is created
- the scope is the mapping of names to objects

```
def f( x ):
    x = x + 1
    print('in f(x): x =', x)
    return x

x = 3
z = f( x )
```

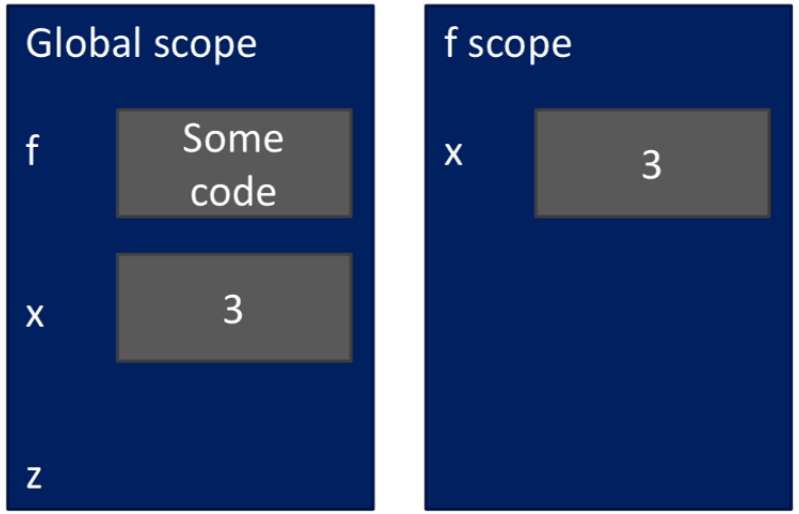

Figure 4: Step 1 of 4.

What's going on here?

1. in the main (i.e., global) scope, we define function `f`
2. in the main scope, we define variable `x`
3. function `f` is called and variable `x` is fed to `f`

```
def f( x ):
    x = x + 1
    print('in f(x): x =', x)
    return x

x = 3
z = f( x )
```

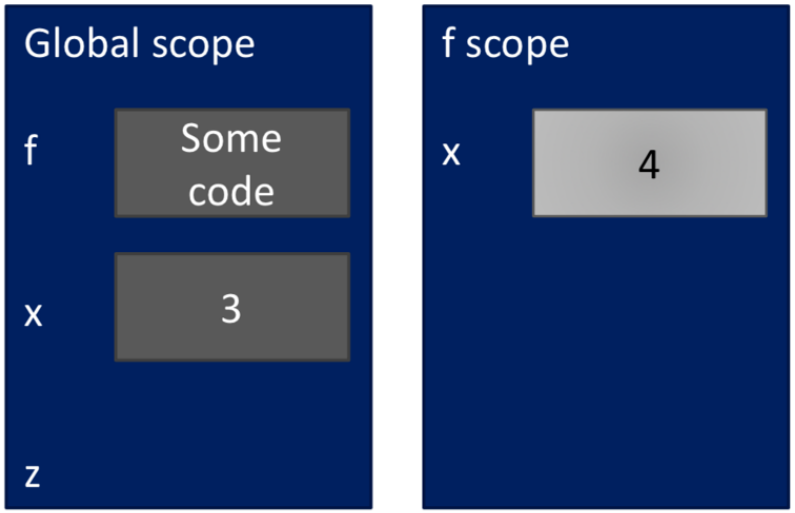

Figure 5: Step 2 of 4.

What's going on here?

1. variable `x` is changed inside `f` because of `x = x+1`

Note

Variable `x` in the global scope has not been changed because `f` has its own scope!

```
def f( x ):
    x = x + 1
    print('in f(x): x =', x)
    return x

x = 3
z = f( x )
```

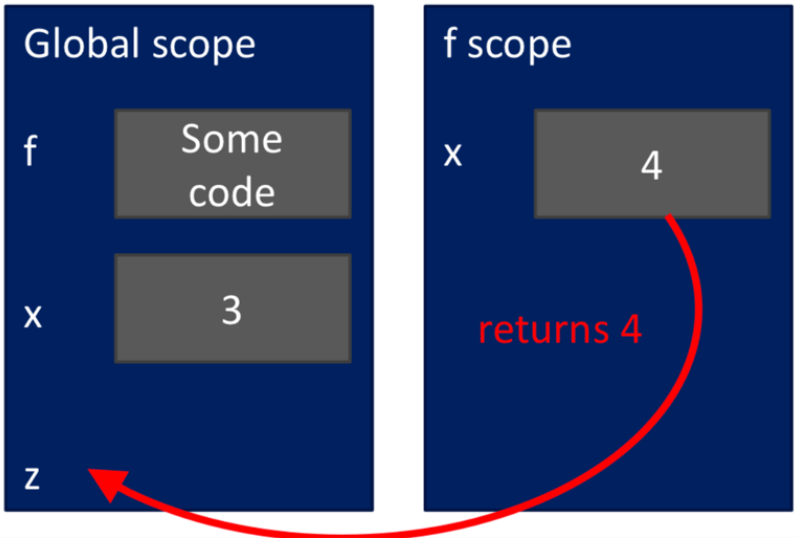

Figure 6: Step 3 of 4.

What's going on here?

1. variable `x` inside `f` is ready to be **`return`**ed
2. in the main scope, there is variable `z` ready to host the **`return`**ed value

```
def f( x ):
    x = x + 1
    print('in f(x): x =', x)
    return x

x = 3
z = f( x )
```

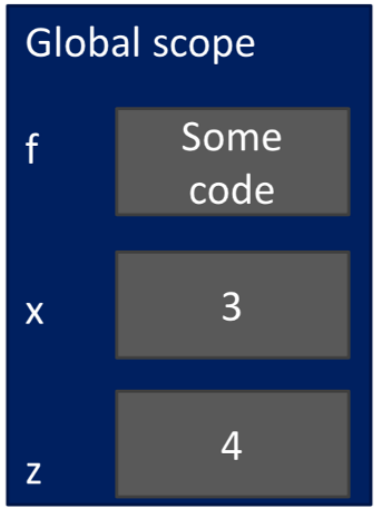

Figure 7: Step 4 of 4.

What's going on here?

1. the frame of function `f` is destroyed because its execution is finished
2. in the main scope, we have the 'original' value for `x`
3. in the main scope, we have the output of function `f` in the new variable `z`

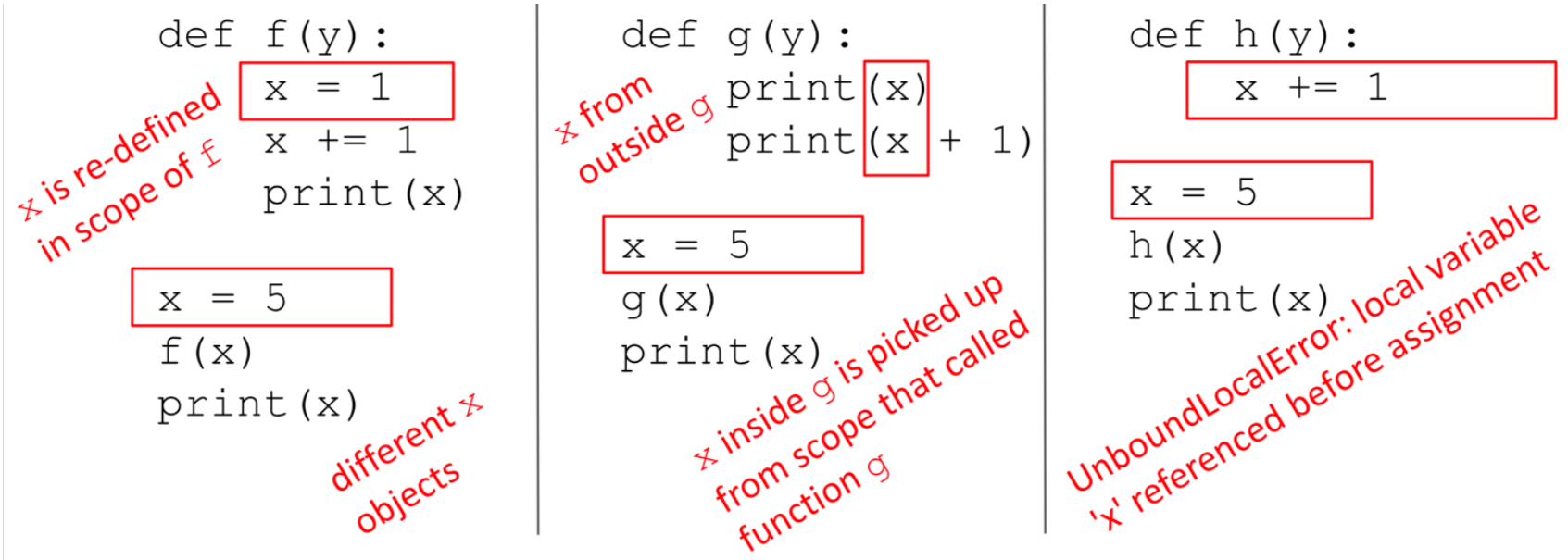

Figure 8: Important caveats (pt.1).

- inside a function, you can access variables defined outside the function itself
- inside a function, you cannot modify variables defined outside the function itself
- to modify variables defined outside, you must use the **global** keyword ... which is frowned upon

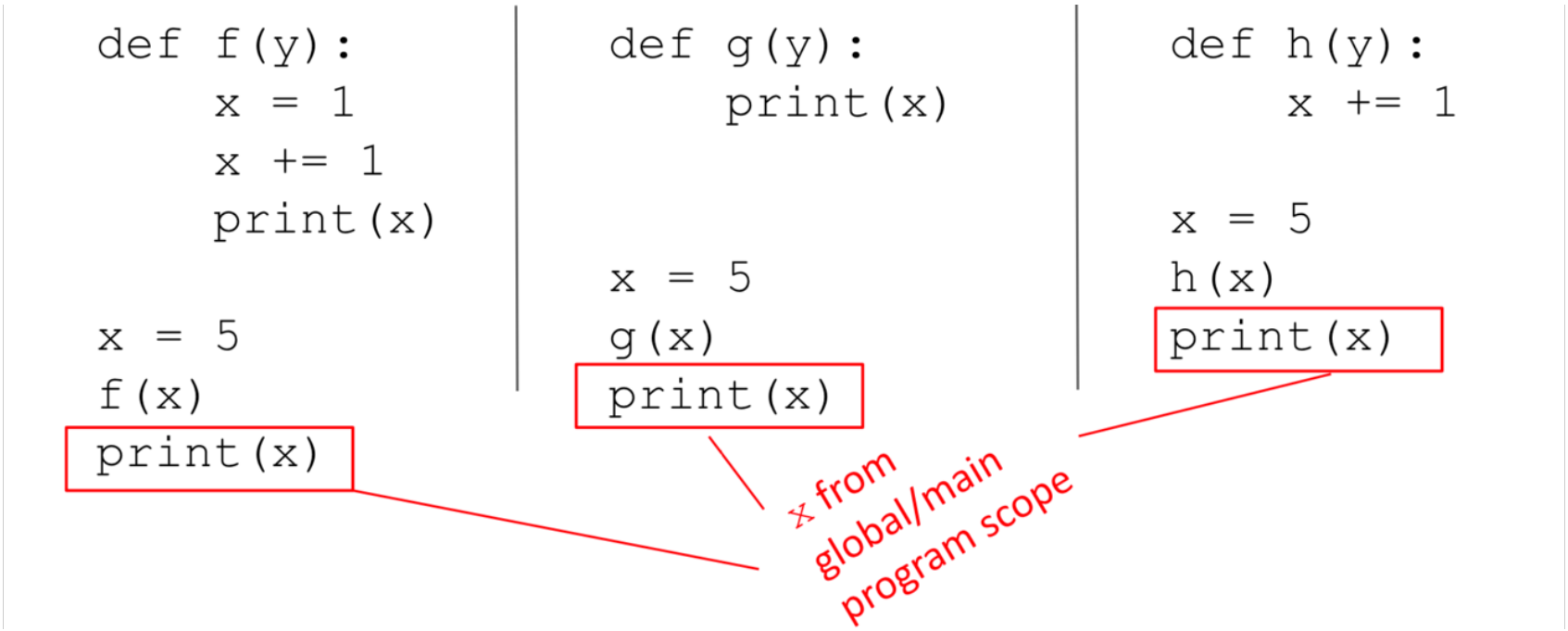

Figure 9: Important caveats (pt.2).

- due to the fact that (by default) you cannot modify variables inside a function, `x` remains untouched

- when we create a variable inside a function, it is local by default
- when we create a variable outside a function, it is global by default: no need to use the **global** keyword
- using the **global** keyword outside a function has no effect
- we use the **global** keyword to *write/modify* a variable inside a function

```
c=0 # global variable

def add():
    global c
    c = c + 2 # increment by 2
    print("Inside add(): c = ", c)

add()
print("In main(): c = ", c)
```

- you can nest functions arbitrarily
- scopes of nested functions behave like Matryoshka dolls
- let's see how thanks to PythonTutor
  `http://pythontutor.com/`

```
def g(x):
    def h():
        x = 'abc'
    x = x + 1
    print('g: x =', x)
    h()
    return x

x = 3
z = g(x)
```

Positional vs. Keyword Arguments

We have seen positional bounding of formal parameters to actual parameters:

- the first formal parameter is bound to the first actual parameter
- the second formal to the second actual
- ... and so on, until all parameters are bounded

Python also supports keyword bounding

- formals are bound to actuals using the name of the formal parameter

Example:

```
def add(a, b, c):
    return a + b + c

add(1,2,3)              # positional bounding a→1, b→2, c→3
add(c=3, a=1, b=2)      # keyword bounding (order does not matter)
```

All parameters need to be provided to a function:

- via positional or keyword arguments
- via default parameters

Example:

```
def area_cylinder(radius=1, height=1):
    pi = 3.14
    bottom_area = pi * radius ** 2
    side_area = pi * height * radius * 2
    return side_area + bottom_area * 2

print("No parameters: ", area_cylinder(), "\n")

print("One parameter: ", area_cylinder(2))
print("Check: ", area_cylinder(2, 1), "\n")

print("Two parameters: ", area_cylinder(2, 3), "\n")

print("Specific parameters: ", area_cylinder(height=2))
print("Check: ", area_cylinder(1, 2), "\n")
```

Built-in Functions

Python implements many functions by default, which are called built-in functions.

Examples of built-in functions:

- numerical functions, such as `round(x)` and `pow(x,y)`[3]
- casting, such as `int(x)`, `float(x)` and `str(x)`
- input and output, namely `input()` and `print()`
- help function, namely `help()`[4]

Related functions are grouped into files, called modules.

[3] Which is another way of doing exponentiation: `pow(x,y)` is the same as `x**y`.

[4] The `help()` function takes as input the name of a function and returns its docstring.

Modules

Modular programming is the way of breaking a large, unwieldy programming task into separate, smaller (and more manageable) subtasks.

A module is a piece of software with a specific functionality.
For example, a game (e.g., chess, ping-pong, ...):

- a module responsible for the game logic (rules)
- a module responsible for graphics

In Python, each module is a Python file with a `.py` extension, yet other extensions are also possible (e.g., pre-compiled C++).

Modules contain some Python code (e.g., functions, class definitions, ...).

An example: the **random** module.

random.py

```
def sample(...):
    """Chooses k unique random elements from a population..."""
    some code
    some code

def randint(...):
    """Return random integer in range [a, b], including both end points."""
    some code
    some code

def random(...):
    """Get the next random number in the range [0.0, 1.0)."""
    some code
    some code
```

Modules are imported (in your script, in your function or in other modules) using the **import** command.

You can use two different strategies for using a specific function from that module.

1. using the **<module>.<function>** notation

```
>>> import random                          → or any other module
>>> print(random.randint(1, 100))          → or any other function
65                                         → your result might vary
```

2. using the **from** command

```
>>> from random import randint             → or any other function
>>> print(randint(1, 100))
65                                         → your result might vary
```

Warning

Strategy #2 does not need the dot-notation (see Strategy #1). Yet, it only loads a specific function in your main!

Python comes with many modules, including:

`io`	read and write from files
`math`	mathematical functions
`random`	pseudo-random number generator
`string`	useful string functions
`sys`	information about your operating system

You can find the full list here:

`https://docs.python.org/3/library/`

An example: the **math** module:

- Number-theoretic and representation functions
- Power and logarithmic functions
- Trigonometric functions
- Angular conversion
- Hyperbolic functions
- Special functions and constants

```
>>> import math
>>> print(math.factorial(x))      # return x!
>>> print(math.log2(x))           # return the base-2 logarithm of x
>>> print(math.cos(math.pi))      # return the cosine of π
>>> print(math.gcd(x,y))          # return the greatest common divisor
```

You can find the full list here:

https://docs.python.org/3/library/math.html

You can browse the contents of a module interactively via the **dir()** function.

```
>>> import math
>>> print(dir(math))
['__doc__', '__file__', '__loader__', '__name__', '__package__',
'__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil',
'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'expm1',
'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum','gamma', 'gcd', 'hypot',
'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma',
'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'nextafter', 'perm', 'pi', 'pow',
'prod', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh',
'tau', 'trunc', 'ulp']
```

Note

Items beginning/ending with '__' are some metadata files. You can safely ignore them. For example, '**__file__**' contains the path pointing to the module source file.

Given a list of functions, you can browse their documentation via the **help()** function.

Recall

The **help()** function takes as input the name of a function and returns its docstring.

```
>>> import math
>>> help(math.log)
Help on built-in function log in module math:

log(...)

    log(x, [base=math.e])
    If the base not specified, returns the natural logarithm (base e) of x.

(END)
```

References

- Guttag, J. V. (2013). *Introduction to Computation and Programming using Python (Revised and Expanded Edition)*. MIT Press. [Chapter 4]
- Downey, A. (2015). *Think Python*. Green Tea Press ed. 2nd. [Chapter 3]

Acknowledgments